

Schatzsuche

Zeitlimit: 8 s Speicherlimit: 256 MiB

Du suchst mit einem magischen Kompass nach einem lange verschollenen Schatz auf einem riesigen Feld aus $N \times N$ Zellen. Insgesamt sind $K \leq 3$ Schatztruhen an verschiedenen Zellen im Feld versteckt. Dein Ziel ist einfach: Finde sie alle!

Wenn du den Kompass auf eine Zelle legst, findet er die kürzesten Wege zu allen Schatztruhen, die am nächsten sind. Dabei werden nur die Bewegungen nach links, oben, rechts und unten verwendet (d. h. die Distanz zu allen nächstgelegenen Schatztruhen gemäss Manhattan-Distanz). Anschliessend zeigt er die Richtung des ersten Schritts an. Falls mehrere gültige erste Schritte zu einer nächstgelegenen Schatztruhe (oder zu mehreren von ihnen) führen, gibt der Kompass alle diese Richtungen zurück. Falls die Zelle einen Schatz enthält, zeigt der Kompass dies stattdessen an.

Immer wenn du eine Schatztruhe findest, leerst du ihren Inhalt, aber du kannst die Truhe nicht entfernen – sie ist zu schwer. Dein Kompass weiss nicht, ob eine Truhe voll oder leer ist. Er zeigt den Weg zu einer (oder falls vorhanden mehreren) nächstgelegenen Truhe, egal ob diese leer oder voll ist.

Versuche, die Position aller Schatztruhen mit möglichst wenigen Kompassabfragen zu finden.

Aufgabe

Dies ist eine interaktive Aufgabe. In jedem Testfall (d. h. bei jeder Ausführung deines Programms) muss dein Programm mehrere Schatzsuchen lösen. Du sollst mit dem Grader über eine bereitgestellte Bibliothek interagieren. Diese Bibliothek stellt die folgenden Funktionen bereit:

- **void** *NextHunt*(**int** &*N*, **int** &*K*) — rufe diese Funktion auf, um die nächste Schatzsuche zu starten. Sie setzt den Parameter *N* auf die Grösse des Gitters und den Parameter *K* auf die Anzahl der Schätze. Falls in der aktuellen Programmausführung keine weiteren Schatzsuchen mehr zu lösen sind, setzt die Funktion *N* und *K* auf -1 ; in diesem Fall solltest du dein Programm mit dem Exit-Code 0 beenden. Beachte, dass du diese Funktion auch aufrufen darfst, bevor du alle Schätze der aktuellen Schatzsuche gefunden hast, z. B. wenn du nur Teilpunkte anstrebst.
- **enum** {*TREASURE* = 0, *DIR_RIGHT* = 1, *DIR_UP* = 2, *DIR_LEFT* = 4, *DIR_DOWN* = 8 }; — dies sind Konstanten, die in den Rückgabewerten der Funktion *Query* verwendet werden (siehe unten).
- **int** *Query*(**int** *x*, **int** *y*) — falls die Zelle an den Koordinaten (*x*, *y*) einen Schatz enthält, gibt diese Funktion *TREASURE* zurück. Andernfalls gibt sie die Summe einer oder mehrerer der Konstanten *DIR_RIGHT*, *DIR_UP*, *DIR_LEFT* und *DIR_DOWN* zurück und zeigt damit an, welche Bewegungen der Kompass liefert, wenn er auf die Zelle (*x*, *y*) gelegt wird. Die Koordinaten *x* und *y* müssen Ganzzahlen von 0 bis $N - 1$ sein. (Hinweis: In dieser Aufgabe wachsen die *y*-Koordinaten von oben nach unten.)

Nachdem *NextHunt* die Werte $N = K = -1$ gesetzt hat, darf dein Programm weder *NextHunt* noch *Query* erneut aufrufen. Es darf ausserdem *Query* nicht vor dem ersten Aufruf von *NextHunt* aufrufen. Falls dein Programm diese Bedingungen nicht einhält, oder falls es mehr als 1000 Abfragen innerhalb einer einzelnen Schatzsuche durchführt, oder falls es Werte für x und/oder y ausserhalb des zulässigen Bereichs an *Query* übergibt, beendet die Bibliothek dein Programm und vergibt für den aktuellen Testfall das Urteil Run-Time Error (RTE).

Ein Schatz gilt als gefunden, wenn du die Zelle, die ihn enthält, mindestens einmal abgefragt hast. Falls dein Programm *NextHunt* aufruft, bevor alle Schätze der aktuellen Schatzsuche gefunden wurden, wird dies nicht als Fehler behandelt, beeinflusst jedoch deine Punktzahl (mehr dazu weiter unten).

Um auf die Bibliothek zuzugreifen, sollte dein Programm die Header-Datei `treasurehuntlib.h` einbinden:

```
#include "treasurehuntlib.h"
```

Du kannst diese Header-Datei hier herunterladen: `treasurehuntlib.h`.

Um dir bei der Entwicklung deiner Lösung zu helfen, steht hier eine einfache Implementierung der Bibliothek zur Verfügung: `treasurehuntlib-public.cpp`. Um sie zusammen mit deinem Programm zu kompilieren, füge ihren Namen einfach als Parameter für den Compiler hinzu, z. B.:

```
g++ foo.cpp treasurehuntlib-public.cpp
```

wenn `foo.cpp` der Name der Datei ist, die deine Lösung enthält.

Die Implementierung in `treasurehuntlib-public.cpp` unterstützt zusätzlich zu den oben genannten Funktionen eine weitere Funktion namens `void InitFromFile(const char *fileName)`, die eine Liste von Schatzsuchen aus einer Datei einliest und es dir ermöglicht, das Spiel mit diesen zu spielen, anstatt zufällige Schatzsuchen zu generieren. Weitere Details findest du in der Datei `treasurehuntlib-public.cpp`.

Auf dem Grader wird eine andere Implementierung der Bibliothek verwendet, daher solltest du keine Annahmen darüber treffen, wie genau die Implementierung funktioniert. Du darfst jedoch annehmen, dass es im globalen Namensraum nur die oben aufgeführten Deklarationen (*NextHunt*, *Query* und die fünf Konstanten) gibt.

Dein Code darf weder von der Standardeingabe lesen noch auf die Standardausgabe schreiben, da diese von der Implementierung der Bibliothek auf dem Grader zur Kommunikation mit der restlichen Bewertungsumgebung verwendet werden.

Eingabe

Beschränkungen

- $2 \leq N \leq 10^6$
- $1 \leq K \leq 3$
- Innerhalb einer einzelnen Programmausführung wird es höchstens 100 000 Schatzsuchen geben.
- Innerhalb einer einzelnen Schatzsuche dürfen höchstens 1000 Abfragen durchgeführt werden.
- Der Grader ist nicht adaptiv.

Teilaufgaben

- Teilaufgabe 1 (10 Punkte) $K = 1$
- Teilaufgabe 2 (30 Punkte) $K = 2$
- Teilaufgabe 3 (60 Punkte) $K = 3$.

Bewertung

Eine Teilaufgabe kann aus mehreren Testfällen (mehreren Ausführungen deines Programms) bestehen, und jeder Testfall kann mehrere Schatzsuchen enthalten. Für die Bewertung werden alle Schatzsuchen einer gegebenen Teilaufgabe gemeinsam betrachtet, unabhängig davon, wie sie ursprünglich auf die Testfälle verteilt waren. Für die i -te Schatzsuche bezeichnen wir die Gittergrösse mit $N_i \times N_i$, die Anzahl der Schätze mit K_i , die Anzahl der von deinem Programm durchgeführten Abfragen mit Q_i und die Anzahl der gefundenen Schätze mit F_i . Weiterhin bezeichnen wir mit S die Gesamtzahl der für diese Teilaufgabe verfügbaren Punkte. Dann ergibt sich die Anzahl der Punkte, die dein Programm für diese Teilaufgabe erhält, wie folgt:

- Falls dein Programm immer alle Schätze gefunden hat (d. h. falls $F_i = K_i$ für alle i gilt), hängt die Punktzahl ab von $t_i = \frac{Q_i}{\lceil \log_2 N_i \rceil}$:

$$\frac{S}{2} + \frac{S}{2} \cdot \min_i f(t_i) \text{ Punkte, wobei } f(t_i) = \begin{cases} 1, & t_i \leq 11 \\ 1 - (t_i - 11)/9, & 11 \leq t_i \leq 20 \\ 0, & t \geq 20. \end{cases}$$

- Falls dein Programm nicht immer alle Schätze gefunden hat (d. h. es existiert ein i mit $F_i < K_i$), erhält es

$$\frac{S}{2} \cdot \min_i \frac{F_i}{K_i} \text{ Punkte.}$$

Mit anderen Worten: Du erhältst die Hälfte der Punkte dafür, alle Schätze zu finden, und die andere Hälfte dafür, sie mit möglichst wenigen Abfragen zu finden. Für die volle Punktzahl sollte deine Lösung alle Schätze in höchstens $11 \lceil \log_2 N_i \rceil$ Abfragen finden. Zwischen $11 \lceil \log_2 N_i \rceil$ und $20 \lceil \log_2 N_i \rceil$ Abfragen sinkt die Punktzahl linear; und falls deine Lösung mehr als $20 \lceil \log_2 N_i \rceil$ Abfragen benötigt, erhält sie nur die erste Hälfte der Punkte für das Finden aller Schätze. (Die Symbole $\lceil \cdot \rceil$ bedeuten, dass der Wert von $\log_2 N_i$ auf die nächsthöhere ganze Zahl aufgerundet wird.)

Falls die obigen Formeln zu einer nicht ganzzahligen Punktzahl für die Teilaufgabe führen, wird diese auf die nächstgelegene ganze Zahl gerundet.

Falls dein Programm einen Laufzeitfehler verursacht oder sich nicht an das oben beschriebene Protokoll zur Nutzung der Bibliothek hält, erhält es 0 Punkte für die gesamte Teilaufgabe. Um Teilpunkte für das Finden nur eines Teils der Schätze zu erhalten, muss dein Programm die Suche daher ordnungsgemäss durch einen Aufruf von *NextHunt* beenden.

Beispiel

Aufruf	Rückgabewert
NextHunt(N, K)	$N = 4, K = 1$
Query(2, 0)	$DIR_DOWN + DIR_RIGHT = 9$
Query(3, 1)	DIR_DOWN
Query(3, 2)	$TREASURE$
NextHunt(N, K)	$N = -1, K = -1$